

PROGRAMA INTEGRAL DE FORMACIÓN EN PYTHON

CURSO 4: TALLER PRÁCTICO & PROYECTO FINAL PERSONALIZADO
(NIVEL 4 (INTEGRADOR))

Track 02: Chatbot Inteligente para Atención al Cliente

«El Recepcionista Omnicanal y el Manual de Operaciones»

Arquitectura de chatbots conversacionales de producción: Integración de LLMs, memoria de sesión multi-usuario, guardrails de seguridad y conexión multicanal (Telegram, WhatsApp, Web).

Instructor: William Rodríguez (Wisrovi)

Rol: AI Solutions Architect & Principal Software Engineer

Nivel del Curso: Integrador / Producción | **Python:** 3.10+ | **Licencia:** MIT

Acerca del Autor y Mentor



William Rodríguez (Wisrovi)

AI Solutions Architect & Principal Software Engineer • Badajoz, España

Ingeniero y arquitecto de software especializado en Inteligencia Artificial Generativa, sistemas multi-agente, Visión por Computador e infraestructuras MLOps de alta disponibilidad. Creador y mantenedor de la suite de software libre **wisrovi SUITE** en PyPI con más de 26 bibliotecas enfocadas en orquestación de pipelines, caching distribuido y optimización de bases de datos.



GitHub: github.com/wisrovi



LinkedIn: [in/wisrovi-rodriguez](https://in.linkedin.com/in/wisrovi-rodriguez)



DockerHub: hub.docker.com/u/wisrovi



Website: wisrovi.dev

Metodología de Aprendizaje: La Regla de la Bicicleta

En este programa no creemos en el aprendizaje pasivo. Programar no se aprende memorizando manuales o mirando videos en segundo plano mientras tomas café; se aprende **escribiendo código**, enfrentando errores de sintaxis, depurando variables y viendo cómo responde el intérprete en tiempo real.



El Compromiso Activo del Estudiante

Abre Visual Studio Code en cada sesión. Escribe cada ejemplo con tus propias manos. Cambia los números, rompe el código deliberadamente para ver el mensaje de error de Python, y luego arréglalo. Ese proceso de experimentación es el que construye sinapsis duraderas.

Estructura Pedagógica de Cada Guía

Cada documento de esta serie está diseñado rigurosamente bajo el estándar de ingeniería de software profesional:

- **Fundamento Conceptual y Metáfora Intuitiva:** Anclaje visual del concepto abstracto a la realidad.
- **Diagrama de Flujo y Arquitectura Mental:** Representación visual de cómo fluye la información.
- **Código de Nivel Profesional:** Sintaxis limpia, comentarios línea a línea y tipado estático (PEP 484).
- **Patrones y Gotchas:** Errores comunes que cometen los desarrolladores y cómo evitarlos.

Índice General de la Sesión

Sección	Contenido y Enfoque Temático	Página
Capítulo 1	Fundamentos y Metáfora Conceptual: Arquitectura de un Chatbot Conversacional de Negocio	Pág. 4
Capítulo 2	Diagrama de Flujo y Arquitectura: Diagrama de Flujo Conversacional y Webhooks	Pág. 5
Capítulo 3	Implementación Práctica en Python: Motor de Chatbot con Historial de Sesión	Pág. 6
Capítulo 4	Patrones Avanzados, Gotchas y Debugging: Gotchas en Chatbots de Producción	Pág. 7
Cierre	Conclusiones, Notas del Mentor y Agradecimiento	Pág. 8
Anexos	Bibliografía Oficial y Enlaces de Profundización	Pág. 9

Objetivos de Aprendizaje (Competencias Clave)



Competencia Conceptual

Comprender la gestión del estado conversacional, el enrutamiento de intenciones y la prevención de alucinaciones corporativas.



Competencia Práctica

Construir un bot conversacional con historial de diálogo, base de conocimiento RAG y despliegue en Telegram o Web.

Requisitos Previos y Entorno Recomendado

Para aprovechar al máximo esta sesión se recomienda tener instalado Python 3.10 o superior, Visual Studio Code con la extensión oficial de Python configurada, y haber completado las lecturas y ejercicios de las sesiones anteriores de la ruta formativa.

1. Arquitectura de un Chatbot Conversacional de Negocio

Un chatbot empresarial no solo charla: responde preguntas frecuentes con precisión quirúrgica, consulta el estado de pedidos y transfiere a humanos cuando es necesario.

Metáfora Central: El Recepcionista Omnicanal y el Manual de Operaciones

El chatbot es como el recepcionista estrella de una empresa: saluda cordialmente, recuerda todo lo que le dijiste en la conversación actual (memoria de sesión) y consulta de inmediato el manual de operaciones antes de dar una respuesta oficial.

Principios Teóricos y Modelo Mental

Gestión de Sesión: Cada usuario tiene un `session_id` único asociado a su buffer de historial en memoria (o en Redis).

Guardrails y System Prompt: Delimitan estrictamente las fronteras temáticas del bot para evitar que hable de temas ajenos a la empresa.

Regla de Oro en Python

Instruye siempre al chatbot en su System Prompt para que admita honestamente si no conoce una respuesta en lugar de inventar información.

2. Diagrama de Flujo Conversacional y Webhooks

Ciclo de vida del mensaje desde la app de mensajería hasta la síntesis de respuesta.



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Intérprete	Estado en Memoria
1. Inicialización	El usuario envía un mensaje en Telegram/WhatsApp; la plataforma emite un Webhook HTTP.	Mensaje entrante
2. Evaluación	El Session Manager recupera el historial previo del usuario desde la memoria caché.	Historial de diálogo cargado
3. Transformación	Se inyecta el contexto RAG de la empresa y el LLM formula la respuesta corporativa.	Inferencia contextualizada
4. Retorno / Salida	Se guarda el nuevo turno en el historial y se envía el mensaje al canal del usuario.	Respuesta entregada al chat

Visualización Mental

Mantén los prompts del sistema concisos y enfócate en el tono de voz (amable, formal, conciso).

3. Motor de Chatbot con Historial de Sesión

Gestor de memoria de diálogo multi-usuario en Python:

main.py (Python 3.10+)

UTF-8 • PEP 8 Compliant

```
class ChatbotAtencionCliente:
    def __init__(self, nombre_empresa: str):
        self.nombre_empresa = nombre_empresa
        self.sesiones: dict[str, list] = {}
        self.system_prompt = f"Eres el asistente virtual de {nombre_empresa}. Sé conciso y formal."

    def responder_usuario(self, user_id: str, mensaje: str) -> str:
        if user_id not in self.sesiones:
            self.sesiones[user_id] = [{"role": "system", "content": self.system_prompt}]

        # Agregar turno del usuario
        self.sesiones[user_id].append({"role": "user", "content": mensaje})

        # Simulación de respuesta del LLM contextualizada
        respuesta = f"Hola, gracias por contactar a {self.nombre_empresa}. ¿En qué puedo ayudarte?"
        self.sesiones[user_id].append({"role": "assistant", "content": respuesta})

        return respuesta
```

Análisis del Código Fuente

Clase que gestiona sesiones independientes por usuario, acumulando el historial en el formato canónico de roles de los LLMs.

4. Gotchas en Chatbots de Producción

Errores comunes en sistemas conversacionales:

⚠ Gotcha Frecuente (Trampa de Principiante)

No limitar el tamaño del historial acumulado; con el tiempo la conversación agota la ventana de contexto y eleva los costes innecesariamente.

Patrón Recomendado vs Antipatrón

✗ Antipatrón / Mal Código

Acumular cientos de mensajes sin podar el historial

✓ Patrón Pythonic / Correcto

Mantener solo los últimos K mensajes (Sliding Window Memory)

🛡 Consejo de Resiliencia en Producción

Usa una ventana deslizante (ej: últimos 10 mensajes) o resume periódicamente los turnos anteriores.

5. Resumen Ejecutivo y Conclusiones

Dominas la arquitectura completa de un agente conversacional inteligente para atención a clientes.



Logro Alcanzado en esta Clase

Capacidad para construir y desplegar chatbots empresariales con memoria y contexto corporativo.

Notas del Instructor y Siguientes Pasos

Presenta este proyecto en tu portafolio como demostración de integración práctica de IA en procesos de negocio.



Mensaje de Agradecimiento

Muchas gracias por tu entusiasmo, disciplina y dedicación al participar en este programa formativo. La programación es un superpoder que transforma vidas cuando se ejerce con constancia y curiosidad. ¡Nos vemos en la próxima sesión para seguir construyendo juntos!

«El código más elegante es aquel que cualquiera puede entender y nadie necesita reescribir.»

6. Fuentes Bibliográficas y Recursos de Estudio

Para profundizar y consolidar los conocimientos adquiridos en esta clase, consulta las siguientes referencias:

Recurso / Fuente	Descripción Temática	Enlace Oficial
Documentación Oficial de Python	Referencia canónica del lenguaje y librería estándar	docs.python.org/3/
PEP 8 - Style Guide for Python	Guía oficial de estilo, formato e indentación	peps.python.org
Real Python Tutorials	Artículos técnicos y patrones de desarrollo moderno	realpython.com
Python Type Checking (PEP 484)	Anotaciones de tipo y análisis estático en Python	docs.python.org/typing
Suite Open Source wisrovi	Paquetes Python para orquestación y alto rendimiento	github.com/wisrovi

Canales de Consulta y Comunidad

Si encuentras dificultades al replicar el código o tienes dudas sobre la arquitectura de los ejercicios, no dudes en abrir un Issue en el repositorio oficial del curso en GitHub o participar activamente en nuestras sesiones grupales de mentoría.



Desafío de Autoestudio Recomendado

Integra la biblioteca python-telegram-bot para publicar tu chatbot en vivo en un canal de Telegram.